separate accounts can work well with the *Command-Query Responsibility Segregation* (CQRS, see https://martinfowler.com/bliki/CQRS.html) API design pattern, in which a completely separate API is used for query operations compared to update operations.

- Many databases support *row-level security* policies that allow queries and updates to see a filtered view of database tables based on contextual information supplied by the application. For example, you could configure a policy that restricts the tokens that can be viewed or updated to only those with a username attribute matching the current API user. This would prevent an attacker from exploiting an SQL vulnerability to view or modify any other user's tokens. The H2 database used in this book does not support row-level security policies. See https://www.postgresql.org/docs/current/ddl-rowsecurity.html for how to configure row-level security policies for PostgreSQL as an example.

EXERCISES

4. Where should you store the secret key used for protecting database tokens with HMAC?

a) In the database alongside the tokens.

b) In a keystore accessible only to your API servers.

c) Printed out in a physical safe in your boss's office.

d) Hard coded into your API's source code on GitHub.